

The Good, The Bad, and The Ugly: The (Gory) Details of an Internal Developer Platform (IDP)



Jarno Virtanen

***Internal Developer Platform (IDP) is built
by a platform team to build golden
paths and enable developer self-service***



**We'll start with an example
from our systems**



**One app team reports that
their service doesn't respond,
at all**



**OK, they've applied pending
IaC changes because their
drift control noticed a change**



**The pending changes were
from the underlying app
module**



**The team didn't change
anything themselves**



```
~ metadata {  
  ~ annotations      = {  
    ~ "alb.ingress.kubernetes.io/scheme"      = "internet-facing" -> "internal"  
      # (15 unchanged elements hidden)  
  }  
}
```



**alb.ingress.kubernetes.io/scheme =
“internet-facing” -> “internal”**



**Yes, might look suspicious for
an Ops engineer, or for an
app dev**



**But if you're applying
changes effectively forced
upon you**



**These feel like details you
shouldn't need to care about**



```
variable "internal" {  
    description = "If true, the app (actually ALB) will be internal  
only"  
    type        = bool  
-   default     = false  
+   default     = true  
}
```





**Dozens of apps had not set
internal=false explicitly**



**=> their load balancer was
changed to *internal* on IaC
apply**



**And our cloud proxy couldn't
communicate with an internal
load balancer**

BOOM!



**The fix was easy, but it took
like 15-20 mins for the service
to get back online**



**Two sides thinking it's the
other side's fault**



IDP/Ops team: “*you*, as the dev, need to understand the changes you apply”



**App dev: “I *don’t* understand
(all) the changes I am
effectively *forced* to apply”**



***What I think* is that this nicely
highlights the problems with
our IDP**



**Simply put, *the API* we
provide is bad**



**Also, the IDP has unclear
division of responsibilities**



**So, next: the good, the bad,
and the ugly**

“So *who* are you anyhow?”



**I work in the Ops team at
Yleisradio Finland**



Yleisradio is the Finnish Broadcasting Company



**We operate and produce a
couple of TV and radio
channels**

yle.fi, the news website

yle

Etusivu

Venäjän hyökkäys

Koripallon EM-kisat

Sanapyramidi

JA

Search

Menu

News

Top stories

Latest

About us

More



Wolt to appeal court's decision deeming couriers as employees

In a decision in May, the Supreme Administrative Court ruled that the company's couriers were employees, not entrepreneurs.

Working life 15:45

Poland's new president visits Helsinki after White House talks

Foreign policy 14:35



Unions sharply criticise budget decisions



Most popular

1

Education

Turku University suspends four students for cheating

2

Crime

Finland to deport controversial rapper Milan Jaff

3

Media

Wednesday's papers: Government agrees belt-tightening budget, Helsinki military drills, and hoop dreams

4

Economy

Unions sharply criticise budget decisions but corporate and taxpayers' groups are pleased

5

Crime

Finnish court sentences Simon Ekpa to prison on terrorism charges

Latest

15:45

Working life

Wolt to appeal court's decision deeming couriers as employees

14:35

Foreign policy

Poland's new president visits Helsinki after White House talks

11:31

Economy



areena.yle.fi, the streaming platform



**Ops/IDP team: 7 persons,
platform users: ~100,
app instances: ~200**

The Good



[pre-recorded demo]



**Sooo, it is *pretty easy* to get a
service online**



Terraform + Kubernetes



**About 1500 lines of stuff,
does a lot of heavy lifting**



yle tool



**In-house Golang software of
about 30k lines of code**



**The cloud is only operated
through this binary (also on
CI/CD)**



The tool does provide a clear separation of responsibilities

User cares



```
→ projects yle --help
Command-line interface for Yle developer platform

Usage:
  yle [flags]
  yle [command]

Available Commands:
  app      Applications deployments and secrets ma
  asu      Assume role on AWS account
  complete yle complete [bash|zsh|fish|powershell]
  deploy   Deploy application
  ds3      Dragonscale Tools
  ec2      EC2 instances and images management
  ocr      yle ocr pull [image:version] to pull d
```

**implementation
details of the
yle tool**

Ops/IDP team cares

User cares



**things
happening in
the cloud**

Ops/IDP team cares



**All in all, you can get an app
running in the cloud easily**

The Bad,



The interface to the user



env
name
eks_state_bucket
eks_state_key
desired_count
docker_image
env_vars
container_port
namespace
cost_center_tag
internal
log_bucket
ssl_certificate_arn
service_account_name
use_lb
uses_authproxy
authproxy_name
use_http2
additional_annotations
prometheus_exporter_port
prometheus_exporter_path

cpu
memory
cpu_limit
memory_limit
health_check_path
dns_aliases
long_scrape_interval
wait_for_rollout
use_external_secrets
use_reloader
iam_policy
deployment_labels
wafv2_acl_arn
node_arch
enable_autoscaling
autoscaling_cpu_target
autoscaling_custom_metric
autoscaling_external_metric
stabilization_window_up
stabilization_window_down
min_replicas
max_replicas

max_skew_per_zone
max_skew_per_node
when_unsatisfiable
pod_disruption_budget_...
readiness_probe
liveness_probe
cognito_config
alb_group_name
volume_mounts
deployment_max_surge
deployment_max_unavailable
require_ondemand
lifecycle_prestop
alb_balancing_algorithm
lb_type
nlb_port
allow_traffic_only_from_cf
allow_traffic_from_infra_account
scrape_metrics_from_service
additional_envs
test_natgw_addresses
prod_natgw_addresses
nodepool
alb_target_type
uid_gid_override
high_load
pod_dns_ndots
host_aliases
github_team



**That's a whopping 72
variables**



**I think that's just *awful* from
the user's perspective**



**OK, you don't have to provide
all that as a user**

6

Not Committed Yet

```
1 module "app" {
2   source = "git::ssh://git@github.com/Yleisradio/yle-aws-
  tes/app"
3
4   env          = var.env
5   name         = var.name
6   namespace    = local.namespace
7   github_team  = var.github_team
8   cpu          = var.cpu
9   memory       = var.memory
10  eks_state_bucket = var.tf_state_bucket
11  eks_state_key   = local.eks_state_key
12  desired_count  = var.desired_count
13  container_port = var.container_port
14  cost_center_tag = var.cost_center_tag
15  health_check_path = var.health_check_path
16  wait_for_rollout = false
17  dns_aliases     = var.dns_aliases
18  internal        = false
19
20  ssl_certificate_arn = module.acm.acm_certificate_arn
21 }
```



**The solution, unfortunately,
is to provide a *template***

User

**Templated
golden path**





**API should only offer options
that the user cares about and
understands**



**“Does it mean that we don’t
offer every possible option
one could imagine wanting?”**



**YES! That's exactly what it
means**

and The Ugly



**Bigger problem is the unclear
division of responsibilities**

Dev cares



**API /
application
options**

app module

**Ops/IDP
team cares**

Dev cares



**things
happening in
the K8s cluster
and the cloud**



**Ops/IDP
team cares**

**We leak these
implementation details**



**API /
application
options**

**app module
implementation
details**

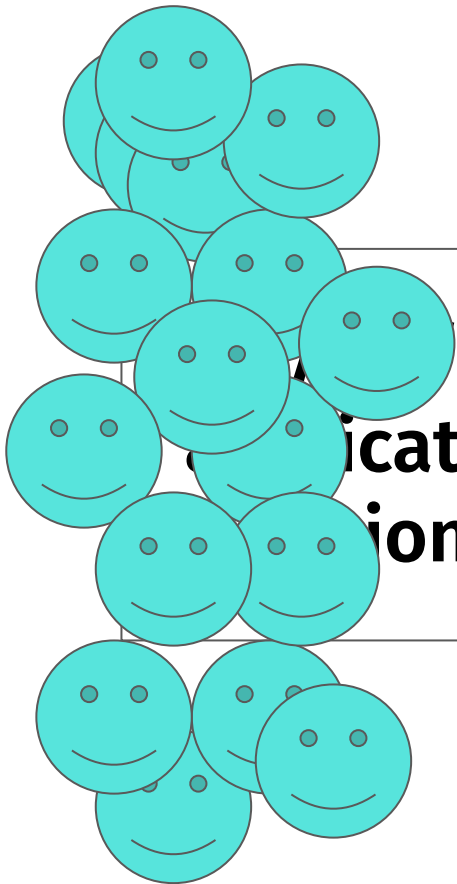
**things
happening in
the K8s cluster
and the cloud**

```
# module.app.kubernetes_deployment.app will be updated in-place
~ resource "kubernetes_deployment" "app" {
  ~ template {
    ~ spec {
      ~ container {
        name = "demo-app"
      ~ readiness_probe {
        ~ initial_delay_seconds = 10 -> 20
      }

      ~ resources {
        ~ limits = {
          ~ "memory" = "384Mi" -> "308Mi"
        }
      }
    }
  }
}
```



**Basically any change that the
app team didn't initiate
themselves**



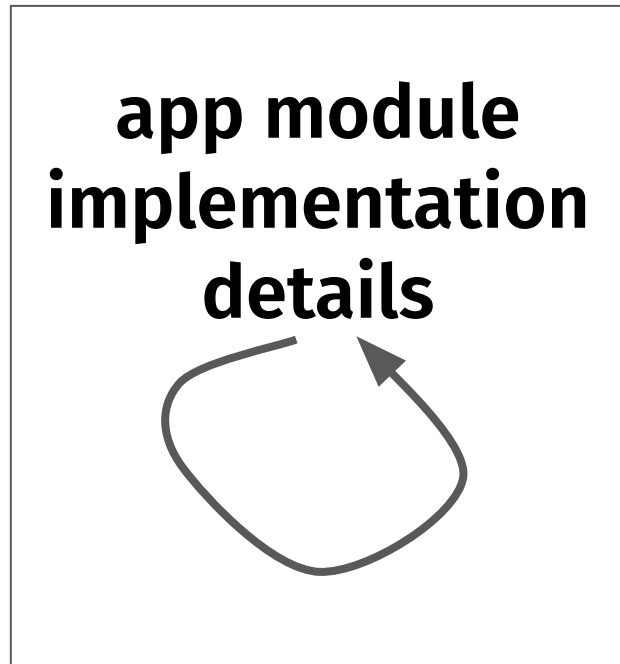
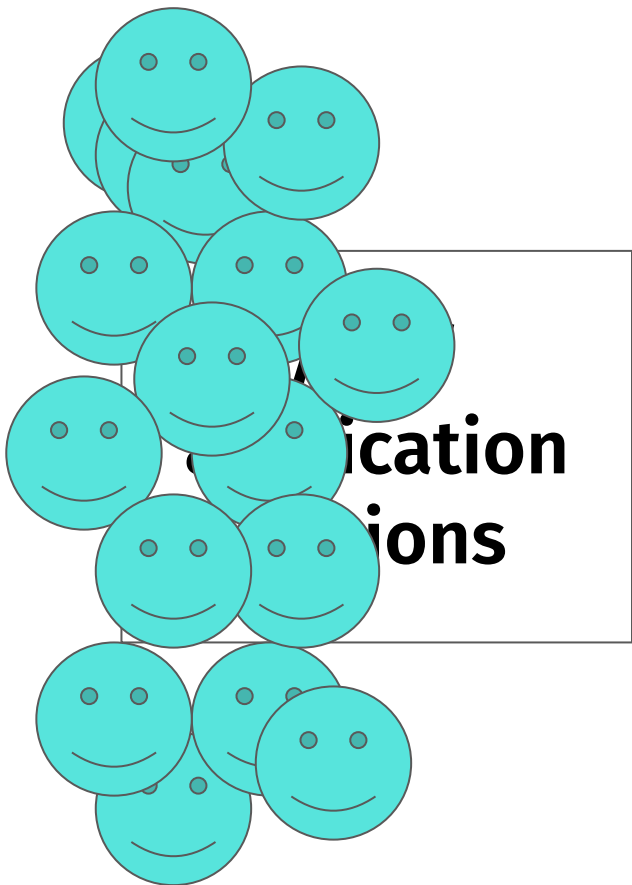
**Application
versions**

**app module
implementation
details**

**things
happening in
the K8s cluster
and the cloud**

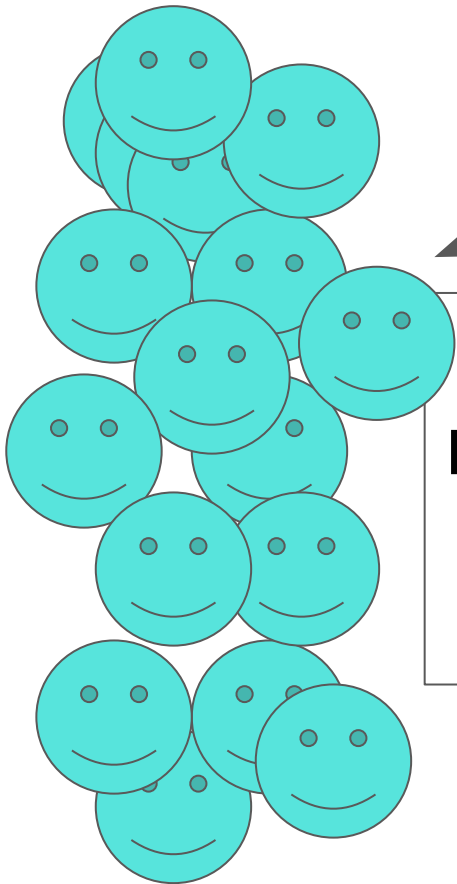
**Users have to apply and understand
the internal details of the module!**

They should only be
visible inside the box





**Imagine you use a 3rd party
payment processor**



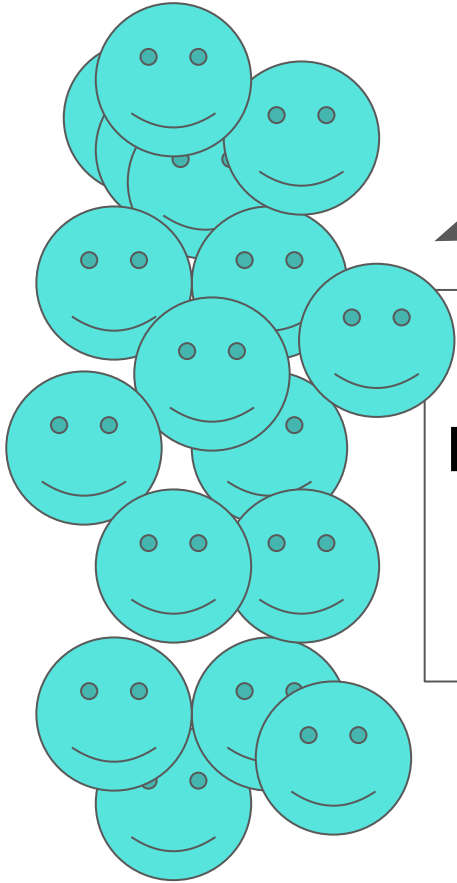
**Payment
API**

**“we changed our internal
implementation, you need to
re-initialize your payment
instance”**

**Payment
Service
implementation
details**

**Some SaaS Payment
processor company**

“after the re-init, X was changed to Y in our systems and that doesn’t work. you applied the change, it’s your fault.”



**Payment
API**

**Payment
Service
implementation
details**



***We force every user/dev to
apply changes they know
nothing about! Or care!***



**So do think of your IDP as it
were service from a separate
company**



**Changes to IDP would be
easier for the IDP team too if
it was properly isolated**



**“What if you just used the
App Engine (du jour)?”**

Dev cares



**API /
application
options**

**App Engine of
some sorts**

**things
happening in
the cloud**

Dev cares



**the end-user
website**



**Sometimes OK, but often we
need the operational
visibility and flexibility**



**In summary: bad APIs lead to
poor UX, overall inefficiency
and more accidents**

**What's the way forward,
then?**





**Easiest way forward is to just
*keep adding more stuff***



**However, we all agree that
that the current model
doesn't work**



**Even if we don't agree about
the reasons**



So, what, then?



**Vendors and lovers of new
shiny tools will tell you that
you need a NEW SHINY TOOL**



Which *might* be true



**But we are already neck-deep
in our current stacks and
other work**



**NEW SHINY TOOL will only
add to that**



K8s operator pattern
approach is tempting

Yle App Operator



**API /
application
options**

**Custom Resource
Definition**

**The old IDP;
runs
Terraform
inside**

**things
happening in
the K8s cluster
and the cloud**



**But this route *is labersome*
and requires a lot of software
engineering**



**There are also tools that
utilize *operator pattern*
smartly (like kro)**



**Whatever the choice, (1) it
must support creating a
good, tight API**



**And (2) the ability to properly
divide responsibilities**

**That is all,
thanks!**



**Kaikille yhteinen, jokaiselle oma
Gemensamt för alla, unikt för mig**